M0 — Free-fall ✅

Status: done — the ported TS replica matches the **real original Nape AS3** bit-for-bit, for mass and all 600 steps.

> The reference is the original engine *executed*: `release_nape.swc` (compiled into `SoccerBalls2.swf`) run under Ruffle, driven by a calling-AS3 harness — not a transpile, not `nape.js`. See the version rule and how it's tested.

What this milestone covers

- **Circle mass properties** — `area = (r·r)·π`, density `raw/1000`, `mass = Σ area·density`.
- **`Space.updateVel`** — gravity + force-based world linear drag (`0.015`).
- **`Space.updatePos`** — position integral (`pos += vel·dt`).
- The whole oracle pipeline: calling AS3 → FFDec inject → Ruffle capture → golden.

The oracle: a calling-AS3 harness driving the real Nape

`tools/nape-oracle/harness-m0.as` replaces the SWF's `Preloader` document class, so the SWF boots straight into a pure Nape test (no game). It builds the exact M0 scenario and `trace()`s each value as raw IEEE-754 bits:

```
var space:Space = new Space(new Vec2(0, 1000));
var mat:Material = new Material(0.3, 0.5, 0.5, 1.0, 0.1);   // density 1.0
var b:Body = new Body(BodyType.DYNAMIC, new Vec2(100, 50));
b.shapes.add(new Circle(12, new Vec2(0, 0), mat));
b.align(); b.space = space;
for (var i:int = 1; i <= 600; i++) { space.step(1.0/60.0, 10, 10); trace("[ORACLE] " + i + " " + bits(b.position.x) + …
```

Captured to `original-goldens/m0-freefall.json` (mass + 600 × `[x,y,vx,vy,rot,angvel]` as 16-hex bit patterns).

The exact test

`src/physics/replica/m0-freefall.test.ts` runs the **same** scenario through the ported replica and asserts every value's bits equal the golden:

```
const got = [w.getX(h), w.getY(h), w.getVX(h), w.getVY(h), w.getRotRad(h), w.getAngVel(h)].map(hex16);
const exp = golden.steps[s];           // captured from the original AS3
// throws on the first field whose bits differ
```

How to run / regenerate

```
# the gate (replica vs the original-AS3 golden)
npx vitest run src/physics/replica/m0-freefall.test.ts --reporter=verbose

# regenerate the golden from the real original (only if harness-m0.as changes)
java -Djava.awt.headless=true -jar tools/vendor/ffdec.jar -replace \
  /Users/jonscott/Projects/SoccerBalls2/bin/SoccerBalls2.swf \
  tools/nape-oracle/m0-oracle.swf Preloader tools/nape-oracle/harness-m0.as
```

```
node tools/nape-oracle/capture-golden.mjs tools/nape-oracle/m0-oracle.swf \
  src/physics/replica/original-goldens/m0-freefall.json
```

Results (verbatim)

```
 ✓ … > circle mass matches the original bit-for-bit
 ✓ … > 600 steps match the original bit-for-bit (x, y, vx, vy, rot, angvel)
 Test Files  1 passed (1)
      Tests  2 passed (2)
```

The golden values (from the original Nape AS3 under Ruffle) that the replica reproduces exactly:

```
mass     3fdcf3f26d5cdffb        (= 0.4523893421169302 = π·12²/1000)
step 1   y=4049238e38e38e39  vy=4030aaaaaaaaaaaa   rot=0  angvel=0
step 600 y=40e74d81ccabf401  vy=40c2239b00b63d88   rot=0  angvel=0
```

`rot` / `angvel` are `0` throughout — free-fall has no `sin/cos`, so AVM2 (original) and V8 (replica) agree to the bit, confirming the float caveat doesn't bite here.

Ported formulas (cited to the original game's decompile)

What	Formula	original source
world drag	`global_lin_drag = global_ang_drag = 0.015`	`ZPP_Space.as:205`
circle area/inertia	`area = (r·r)·π` ; `inertia = r·r·0.5 +	localCOM
density	`Material` stores `density / 1000`	`Material.as:51`
body mass	`cmass = Σ area·density` ; `imass = 1/mass` ; `gravMass = cmass`	`ZPP_Body.as:321/481/537`
velocity integral	`f = force + gravity·gravMass + vel·(−drag·mass)` ; `vel += f·(dt·imass)`	`ZPP_Space.as:1312`
position integral	`pos += vel·dt` ; `rot += angvel·dt`	`ZPP_Space.as:1353`